SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
Re-Accredited by NAAC with 'A++' Grade

Name: Aashray Lal Kayastha

PRN:250770122005

Division: A1

Batch:2025-2029

Sem:3

Subject: Programming with Java
ASSIGNMENT No :3
TITLE : Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

Problem Statement

Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

Learning Objectives

To implement method overloading and use static variables and methods in Java programs.

Theory

Method overloading allows multiple methods to have the same name with different parameter lists, enabling compile-time polymorphism. The static keyword is used for variables and methods that belong to the class instead of individual objects. Static members share common data among all objects and reduce memory usage. These concepts improve code flexibility, readability, and efficient resource management.

Outcome:

Source Code:

```java
class Counter{
    static int count =0; //Static Variable

    Counter(){
        count++; //Increment static variable
        System.out.println("Object Created. Current Count: "+count);
    }
}

public class counter{
    Run main | Debug main | Run | Debug
    public static void main(String[] args){
        Counter obj1=new Counter();
        Counter obj2=new Counter();
        Counter obj3=new Counter();

        System.out.println("Final Count: "+Counter.count); //Access using class name
    }
}
```

Output:

```
C:\Users\Aashray\Documents\University\3rd sem\JAVA>java counter.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Object Created. Current Count: 1
Object Created. Current Count: 2
Object Created. Current Count: 3
Final Count: 3
```

Conclusion

This experiment demonstrated the implementation of method overloading and the use of static variables and methods in Java. Method overloading provides flexibility by allowing multiple methods with the same name but different parameters, while static members enable efficient sharing of data and functionality across all objects of a class. These concepts improve code reusability, organization, and overall program efficiency.

Exercise

1. Develop a Calculator program using overloaded methods for addition of integers and decimals. Use a static variable to count calculations.

Source code:

```
class Calculator {
    // Overloaded method for decimals
    double add(double a, double b) {
        count++;
        return a + b;
    }
}

public class exp3_a {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {

        Calculator cal = new Calculator();

        System.out.println("Integer Addition = " + cal.add(a: 10, b: 20));
        System.out.println("Decimal Addition = " + cal.add(a: 10.5, b: 20.5));

        System.out.println("Total Calculations = " + Calculator.count);
    }
}
```

Output:

```
C:\Users\Aashray\Documents\University\3rd sem\JAVA>java exp3_a.java
Integer Addition = 30
Decimal Addition = 31.0
Total Calculations = 2
```

2. Develop a Restaurant Billing Application where overloaded methods calculate bills for dine-in, takeaway, and delivery orders, while static variables track total orders.

Source Code:

```java
class Restaurant {

    // Static variable to track total orders
    static int totalOrders = 0;

    // Dine-in bill
    double bill(double amount) {
        totalOrders++;
        return amount;
    }

    // Takeaway bill (with packing charge)
    double bill(double amount, double packingCharge) {
        totalOrders++;
        return amount + packingCharge;
    }

    // Delivery bill (with packing and delivery charges)
    double bill(double amount, double packingCharge, double deliveryCharge) {
        totalOrders++;
        return amount + packingCharge + deliveryCharge;
    }
}

public class exp3_b {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {

        Restaurant r = new Restaurant();

        System.out.println("Dine-in Bill = " + r.bill(amount: 500));
        System.out.println("Takeaway Bill = " + r.bill(amount: 500, packingCharge: 20));
        System.out.println("Delivery Bill = " + r.bill(amount: 500, packingCharge: 20, deliveryCharge: 50));

        System.out.println("Total Orders = " + Restaurant.totalOrders);
    }
}
```


Output:

```
C:\Users\Aashray\Documents\University\3rd sem\JAVA>java exp3_b.java
Dine-in Bill = 500.0
Takeaway Bill = 520.0
Delivery Bill = 570.0
Total Orders = 3
```